MONASH University

Practice Test 2 - FIT1045 2025

For the test you will need to bring writing material (pen/pencil), a photo id, and your student card or M-PASS.

The following details will help ensure that we can match your attempt to your account in OnTrack and get the test back to you in your applied class.

Student name: _____

Student ID: _____

Applied class day and time: _____

For this test you:

- must work alone.
- must not use any reference materials.
- will have 60 minutes to complete it.
- must write all answers within this test booklet.
- must hand back your test booklet at the end of the test session.
- must not take any notes or remove any test materials from the test location.
- must place all smart devices away as instructed.
- must not leave in the last 10 minutes of the test.

Your test will be graded as Complete where everything is at the expected standard, Fix and resubmit if you demonstrate competency but made some small mistakes you can correct in your applied class, or Redo if you need to resit the test. Results will be updated in OnTrack for Week 8 and Week 10 test versions.

Official Use		
☐ Complete	☐ Fix and resubmit	☐ Redo

Remember the test will be on paper, so make sure to practice on paper as well as coding up things in Visual Studio Code.

1) Write a sentence to define each of the programming terminology / statements listed below.

1: Local Variable

2: Global Variable

3: Parameter

4: Argument

5: Function

6: Procedure

7: Return

8: Exception

9: Struct

10: Enumeration

11: Array

2) Hand execute the following code within the space below. Ensure you follow the process and format described for doing this in the Field Guide.

```
int first[5] = {1, 1, 3, 4, 5};
int first_len = 5;

int second[3] = {2, 2, 3};
int second_len = 3;

int result[5] = {}; // 0 all values
int len = 5;

int i = 0, j = 0, k = 0;
while (i < first_len && j < second_len && k < len)
{
    if (first[i] <= second[j])
    {
        result[k] = first[i];
        i++;
    }
    else
    {
        result[k] = second[j];
        j++;
    }
    k++;
}

while (i < first_len && k < len)
{
    result[k] = first[i];
    k++;
    i++;
}

while (j < second_len && k < len)
{
    result[k] = second[j];
    k++;
    j++;

}
```

The following scenario applies to the remaining questions.

Student Details: This program will allow the user to record student data, where each student has a name, an id (integer), and a score (integer). The user will interact with this program via the Terminal.

Limit the use of additional functions and procedures to only the following. Assume all necessary code is provided to access these:

- string read_string(string prompt);
- int read_integer(string prompt);
- double read_double(string prompt);
- bool read_boolean(string prompt); // reads Y/N returns true/false
- void write(...);
- void write_line(...);
- string to_string(...);

Here is an example execution of this program (user input in bold):

```
Add a Student: Y
Enter ID: 2
Enter Name: Fred
Enter Score: 52

Add a Student: Y
Enter ID: 5
Enter Name: Wilma
Enter Score: 185
Please enter a value between 0 and 100
Enter Score: 85

Add a Student: Y
Enter ID: 11
Enter Name: Betty
Enter Score: 72

Add a Student: N

Students:
Fred (2) 52
Wilma (5) 85 congratulations
Betty (11) 72

Average Score: 69.666666667
```

This program needs to demonstrate effective use of arrays, structs, control flow, function and procedure creation and use, references, and parameters.

3) Write the C++ code to declare a **student data** struct to model a student:

4) The program will need to work with up to 100 students. Write the C++ code to declare a struct that contains a fixed size array of the type you created in Q3 and a field to track the currently used elements. Outside of this struct, create any constant as needed.

5) In the space below, write the C++ code for a procedure to output a student to the terminal. Output in the format "name (id) score" followed by " congratulations" if the student's score is larger than or equal to 80.

6) In the space below write a C++ function to read a student in from the terminal. Ensure that the score is between 0 and 100 (inclusive) asking the user to re-enter until it is valid.

7) In the space below write a C++ function to calculate the average score from a collection of students (using your struct from Q3). It will be passed the collection as a parameter, and return the mean of the score of the students currently stored in the array.

8) In the space below write a C++ procedure to add a student to the end of a collection of students (using your struct from Q3). It will be passed a reference to the collection as a parameter, and details of the student to add. It will output the message "System Full" and ignore the data if the array is already at capacity.

9) Write the C++ code to implement **main** that will
   - Declare a collection of students using your struct from Q4.
   - Loop while the user wants to add a student to the collection
     i) Read a student in from the user – using your function from Q6
     ii) Add it to the collection – using your procedure from Q8
   - Print all the students from your collection – using your procedure from Q5
   - Print the average score of the students in your collection using your function from Q7.